

课程笔记

斯坦福 CS336 讲座 8：并行化

五道口纳什 & Codex

2026 年 6 月 8 日



视频作者/频道：Stanford CS336

发布日期：2026 年 5 月 21 日

视频时长：01:20:10

视频链接：<https://www.bilibili.com/video/BV12gLx6xEcq?p=8>

目录

1 引言与并行化基础回顾	3
1.1 课程概述与并行化动机	3
1.2 两大瓶颈：算力与内存	3
1.3 节点内与节点间通信	3
1.4 集合通信原语回顾	4
1.5 本章小结	4
2 硬件架构差异：TPU 与 GPU 的网络拓扑	4
2.1 TPU 的环形网格拓扑	4
2.2 GPU 的交换机连接拓扑	5
2.3 硬件趋同与中国的实践	6
2.4 本章小结	7
3 数据并行 (Data Parallelism)	7
3.1 基本原理	7
3.2 通信开销分析	7
3.3 内存占用分析	8
3.4 ZeRO：数据并行的内存优化	8
3.4.1 ZeRO Stage 1: 优化器状态分片	9
3.4.2 ZeRO Stage 2: 梯度分片	9
3.4.3 ZeRO Stage 3: 参数分片 (FSDP)	9
3.5 批量大小与收敛性的权衡	10
3.6 本章小结	11
4 张量并行 (Tensor Parallelism)	11
4.1 动机与核心思想	11
4.2 列切分与行切分	11
4.2.1 列切分 (Column-wise Parallelism)	11
4.2.2 行切分 (Row-wise Parallelism)	12
4.3 前向与反向传播的对偶性	12
4.4 通信量分析	13
4.5 本章小结	14
5 流水线并行 (Pipeline Parallelism)	14
5.1 基本思想	14
5.2 气泡问题与微批次	15
5.2.1 微批次 (Micro-batches)	15
5.2.2 气泡占比的定量分析	16
5.3 1F1B 调度策略	16
5.3.1 核心思想	16
5.3.2 零气泡流水线 (Zero Bubble Pipeline)	16
5.4 流水线并行的通信特性	17

5.5	本章小结	17
6	内存优化策略	18
6.1	内存占用的真实构成	18
6.2	激活值内存的估算	18
6.3	张量并行对激活内存的削减	19
6.4	序列并行 (Sequence Parallelism)	19
6.5	激活值重计算 (Gradient Checkpointing)	20
6.5.1	基本原理	20
6.6	ZeRO 优化器的三阶段回顾	20
6.7	本章小结	21
7	专家并行与 MoE	21
7.1	MoE 架构回顾与专家并行的动机	21
7.2	专家并行 vs. 张量并行：为何优先选择 EP?	22
7.2.1	矩阵粒度与 GPU 利用率	22
7.2.2	稀疏路由 vs. 密集切分	22
7.3	专家并行的通信模式：AllToAll 与低延迟调度	22
7.4	专家并行的工程挑战与前沿库	23
7.4.1	DeepSeek 的 DeepSeek EP	23
7.4.2	NVIDIA 的 Hybrid EP	23
7.5	专家并行与其他并行策略的组约束	23
7.5.1	朴素组合：EP 与 DP 共享复制组	23
7.5.2	注意力层与 MLP 层的并行需求矛盾	23
7.5.3	解决方案：解耦的张量并行	24
7.6	上下文并行 (Context Parallelism)	24
7.7	本章小结	24
8	注意力层并行化与综合策略	25
8.1	从单一策略到多维组合	25
8.2	计算-通信权衡的定量分析	25
8.2.1	FSDP 的扩展极限	26
8.2.2	叠加策略让曲线右移	26
8.3	实操指南：如何选择并行策略	26
8.3.1	序列并行的定位	27
8.4	Megatron 论文的量化证据	27
8.4.1	张量并行的最优规模	28
8.4.2	流水线并行对批次大小的要求	28
8.4.3	激活重计算的反直觉价值	28
8.5	本章小结	29
9	实际大规模训练案例与总结	29
9.1	案例研究：不同模型的并行策略	29
9.1.1	OLMo：纯 FSDP 的小模型训练	29

9.1.2	DeepSeek V1 与 V3: 从稠密到 MoE 的演进	29
9.1.3	Qwen: 经典的三维并行	29
9.1.4	LLaMA 3: 超大规模稠密模型的标杆	30
9.1.5	Gemma 2: TPU 上的简洁方案	30
9.1.6	Mixtral 8x22B: MoE 的经典配置	30
9.1.7	其他前沿模型	30
9.2	NVIDIA Megatron-LM 的推荐配置	31
9.3	各模型配置的共同规律	31
10	总结与延伸	31
10.1	讲座核心要点回顾	31
10.2	核心机制的结构化提炼	33
10.3	各节之间的交叉联系	34
10.4	实践启示与下一步	34
10.5	展望	34

1 引言与并行化基础回顾

1.1 课程概述与并行化动机

现代语言模型的训练规模已经达到了前所未有的程度，单块加速器（GPU/TPU）的算力和内存已远远无法满足需求。本讲座将深入探讨在巨型集群上训练超大模型时所涉及的各种并行化策略，包括数据并行、张量并行、流水线并行、专家并行等，以及它们在实际大规模训练中的组合运用。

核心目标

本讲座的核心目标是理解：当模型规模超出单卡容量时，如何通过多种并行策略的组合，在保持训练无损（数学等价于单卡训练）的前提下，充分利用数据中心级别的算力和内存资源。

讲座最后会介绍一种称为**四维并行化**（4D Parallelism）的策略组合框架——即同时运用四种或更多并行维度，以实现最大规模的并行扩展。这也是实际大规模训练（如 GPT-3、LLaMA 等）所采用的标准做法。

1.2 两大瓶颈：算力与内存

并行化技术的出现，本质上是为了解决两个核心瓶颈：

1. **算力瓶颈 (Compute Bottleneck)**：单块芯片能提供的浮点运算能力远远不够。当前世界上最快的超级计算机已达到**百亿亿次浮点运算**（ExaFLOP）级别，而单个 GPU 与此相差甚远。要训练一个千亿参数级别的语言模型，必须将计算任务分布到成百上千个加速器上。
2. **内存瓶颈 (Memory Bottleneck)**：模型参数本身、梯度、优化器状态以及前向传播中的激活值（Activations）共同占用的内存，远远超出单块 GPU 的显存容量。以 Adam 优化器为例，每个参数需要存储参数本身、梯度、一阶矩估计和二阶矩估计，总计约 4 倍参数量的数据。

常见误解

许多初学者认为并行化只是为了加速训练。实际上，对于当前的大语言模型而言，**内存扩展**往往比算力扩展更为迫切——因为模型根本放不进单块 GPU，不并行化就无法开始训练。

1.3 节点内与节点间通信

在分布式训练中，一个至关重要的概念是**节点内**（Intra-node）与**节点间**（Inter-node）通信的区别：

- **节点内通信**：同一台服务器内部的 GPU 之间通过 NVLink/NVSwitch 等超高速互联，带宽极高（可达数百 GB/s），延迟极低。因此可以在节点内执行通信成本较高的操作（如频繁的全规约）。
- **节点间通信**：不同服务器之间通过 InfiniBand 或以太网连接，带宽显著降低（通常为几十 GB/s 或更低），延迟也更高。这要求节点间的并行策略必须更加节约通信带宽。

为什么节点内/节点间区别如此重要？

这一区别直接决定了并行策略的选择。例如，张量并行（Tensor Parallelism）需要极高的通信带宽，因此通常只在节点内部署；而数据并行（Data Parallelism）的通信量相对可控，可以跨节点扩展。后续的并行策略设计都必须围绕这一硬件约束展开。

1.4 集合通信原语回顾

本讲座中所有并行算法都在**集合通信**（Collective Communication）的抽象层面讨论，而非底层数据包的发送细节。以下是几个核心原语：

- **AllReduce**：将所有设备上的张量求和，并将结果分发回每个设备。每个设备最终获得全局求和结果。
- **AllGather**：收集所有设备上的分片数据，使每个设备都获得完整的拼接结果。
- **ReduceScatter**：先对所有设备的数据求和，再按设备分发各自对应的分片结果。

关键等价关系

AllReduce 与 ReduceScatter + AllGather 在通信成本上是等价的。这一等价关系在后续分析 ZeRO 优化器时至关重要——它允许我们用两个步骤的组合来替代 AllReduce，从而在几乎不增加通信开销的前提下实现内存优化。

数学上，对于 N 个设备，AllReduce 的通信量为 $2(N-1)/N \cdot S \approx 2S$ （ S 为数据大小），而 ReduceScatter + AllGather 的总通信量同样约为 $2S$ 。

1.5 本章小结

本节回顾了并行化的两大驱动力——算力扩展与内存扩展，并强调了节点内/节点间通信带宽差异对并行策略选择的决定性影响。后续所有并行算法都将建立在集合通信原语的抽象之上，其中 AllReduce 与 ReduceScatter+AllGather 的等价关系是理解 ZeRO 等内存优化技术的核心钥匙。

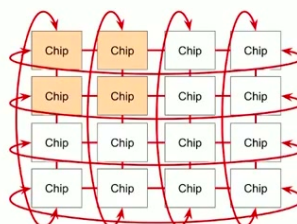
2 硬件架构差异：TPU 与 GPU 的网络拓扑

本节讨论 TPU 与 GPU 在网络连接方式上的根本差异。虽然大部分并行算法在概念上与硬件无关，但底层网络拓扑直接影响了不同训练平台（Google、Meta、OpenAI 等）选择并行策略的倾向。

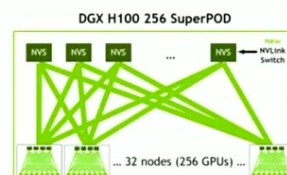
2.1 TPU 的环形网络拓扑

TPU（Tensor Processing Unit）是 Google 开发的专用 AI 加速器。其网络拓扑采用**环形网络**（Torus）结构：

- 每个 TPU 芯片只与相邻的芯片直接相连；
- 网络的两端首尾相连，形成环状结构（实际为 3D Torus）；
- 无论网络规模多大，每个节点的邻居数量基本保持不变。



The diagram illustrates the DGX A100 256 SuperPOD architecture. It features a multi-tier network topology. At the top, there are three blue squares representing 'IB HDR spine switches'. Below them are three blue squares representing '... IB HDR leaf switches ...'. At the bottom, there are three green server racks representing '... 32 nodes (256 GPUs) ...'. The spine switches are connected to the leaf switches, and the leaf switches are connected to the server racks. The connections between the spine and leaf switches are shown as a mesh, indicating a highly interconnected network.



简单说, TPU 的网络是一种环形网格

In one slide, the TPU network is what's called a toroidal mesh.

the slide, the GPU network is called a mesh.

图 1: TPU 环形网格 (Torus) 拓扑示意图。每个芯片只与相邻芯片相连, 邻居之间首尾相连。¹

环形网格的优势

Torus 拓扑的设计哲学是极简与可扩展性：

- 网络结构简单，易于大规模扩展；
- 在相同功耗预算下，每条连接的带宽可以做得很高；
- 对于通信模式规则、可预测的算法（如密集矩阵乘法），性能极佳。

2.2 GPU 的交换机连接拓扑

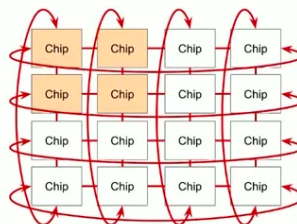
GPU 集群的网络设计则走向另一个极端——**全互联** (Full-Connectivity) 导向的胖树 (Fat-Tree) 结构:

- 底层 GPU 之间通过 NVLink/NVSwitch 实现超高速连接；
- 多个 GPU 组成一个 Pod，Pod 之间通过主干交换机互联；
- 随着节点数量增加，网络规模扩大，拓扑复杂度上升。

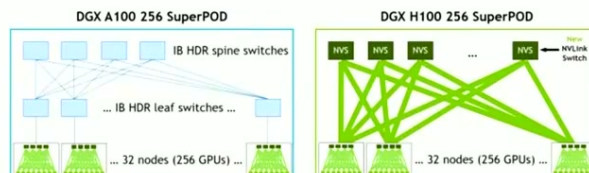
¹视频画面时间区间：00:05:06—00:05:10。

TPUs vs GPUs – design differences at the comm level

TPU networking
toroidal mesh



GPU networking
All-to-all (up to 256)



相比之下，GPU 的网络设计理念则截然不同

In contrast, the GPU networking philosophy is very different.

Contrast, the GPU networking philosophy is very different.

4100 SuperPod		H100 SuperPod		Speedup	
PFPLOP/s	(GB/s)	PFPLOP/s	(GB/s)	Bisection	Reduce
37.6G/s	56 GPUs	80	6,400	100	512
				1x	3x
				1.5x	4.5x

图 2: GPU 胖树 (Fat-Tree) 拓扑示意图。通过多层交换机实现设备间的全互联, 适合不规则通信模式。²

两种拓扑的本质差异

- **TPU (Torus)**: 适合通信模式规则、可预测的工作负载。如果算法只需与邻居通信，TPU 性价比极高。
- **GPU (Fat-Tree)**: 适合通信模式不规则、需要任意节点间高速通信的工作负载。对于路由不确定的算法（如 MoE 的专家并行），GPU 网络更加灵活。

2.3 硬件趋同与中国的实践

有趣的是，TPU 和 GPU 的设计理念正在发生**趋同演化**（Convergent Evolution）：

- Google 最新发布的 TPU v8i 和 TPU v8t 开始采用更接近全互联的树形拓扑，以适应 MoE 模型中大量不规则通信的需求；
- 同时，GPU 集群也在持续增加节点间带宽（如 InfiniBand 的升级）。

这背后的根本原因是：工作负载在定义我们需要的网络。随着语言模型普遍采用 MoE 架构，对全互联带宽的需求推动了硬件设计的收敛。

华为F腾 (Ascend) 的取舍

华为 Ascend 910 采用了一种有趣的架构选择：单颗芯片性能远低于 H200，但通过大规模光纤交换机将多达 384 颗芯片全互联。这种设计用**功耗换取通信自由度**——整体功耗是同等规模 NVIDIA 系统的四倍，但可以用暴力横向扩展来弥补单芯片性能的不足。这体现了硬件设计中“功耗-性能-通信”三角权衡的另一种极端解法。

²视频画面时间区间：00:05:52-00:05:56。

2.4 本章小结

TPU 的环形网格和 GPU 的胖树拓扑代表了两种截然不同的设计哲学：前者追求极简、规则通信下的高效率；后者追求任意节点间通信的灵活性。理解这一差异有助于解释为什么不同训练平台（Google 偏爱密集模型 +TPU，Meta/OpenAI 偏爱 MoE+GPU）会选择不同的并行策略。而随着 MoE 成为主流，两种硬件架构正在向全互联方向趋同。

3 数据并行 (Data Parallelism)

数据并行是最直观、最容易理解的并行策略，也是当前分布式训练的标准做法。其核心思想是：将训练数据分布到多个 GPU 上，每个 GPU 保存一份完整的模型副本，各自计算梯度后同步更新。

3.1 基本原理

考虑标准的随机梯度下降 (SGD)：设全局批次大小为 B ，每个更新步骤取 B 个样本，计算梯度总和后更新参数。

数据并行的做法是将批次拆分为 M 份（ M 为 GPU 数量），每份大小为 $b = B/M$ ：

1. 每个 GPU i 接收数据子批次 b_i ，执行前向传播和反向传播，计算局部梯度 $\mathbf{g}_i$ ；
2. 通过 AllReduce 将所有局部梯度求和，得到全局梯度 $\mathbf{g} = \sum_{i=1}^M \mathbf{g}_i$ ；
3. 每个 GPU 使用全局梯度更新本地参数副本。

```
1 # 每个 GPU 执行相同的代码
2 for batch in dataloader:
3     local_batch = scatter(batch, world_size) # 分发数据
4     loss = model(local_batch)
5     local_grad = autograd(loss, model.parameters())
6
7     # AllReduce: 全局梯度同步
8     global_grad = all_reduce(local_grad, op=SUM)
9
10    # 本地参数更新
11    optimizer.step(global_grad)
```

Listing 1: 数据并行的伪代码示意

3.2 通信开销分析

数据并行的通信开销主要来自梯度同步：

- 每个批次需要通信的数据量约为**参数数量的两倍**（一次发送梯度、一次接收聚合结果）；
- 从计算扩展性角度看，只要每个 GPU 拥有足够的本地样本，数据并行几乎是完美的线性加速；
- 但从内存扩展性角度看，**数据并行完全不节省内存**——每个 GPU 都保存了完整的模型副本、梯度和激活值。

数据并行的内存陷阱

初学者常误以为增加 GPU 数量就能训练更大的模型。实际上，标准数据并行下每个 GPU 仍然需要容纳完整的模型、梯度和优化器状态。如果模型放不进单卡，仅靠数据并行无法解决问题——必须引入模型并行或 ZeRO 等内存优化技术。

3.3 内存占用分析

训练时的内存占用远比参数本身大得多。以 Adam 优化器为例，每个参数大约需要以下存储：

- **参数**（蓝色）：模型权重，通常以 FP16/BF16 存储；
- **梯度**（橙色）：与参数同等大小；
- **优化器状态**（绿色）：Adam 的一阶矩（momentum）和二阶矩（variance），通常以 FP32 存储，占用内存最大。

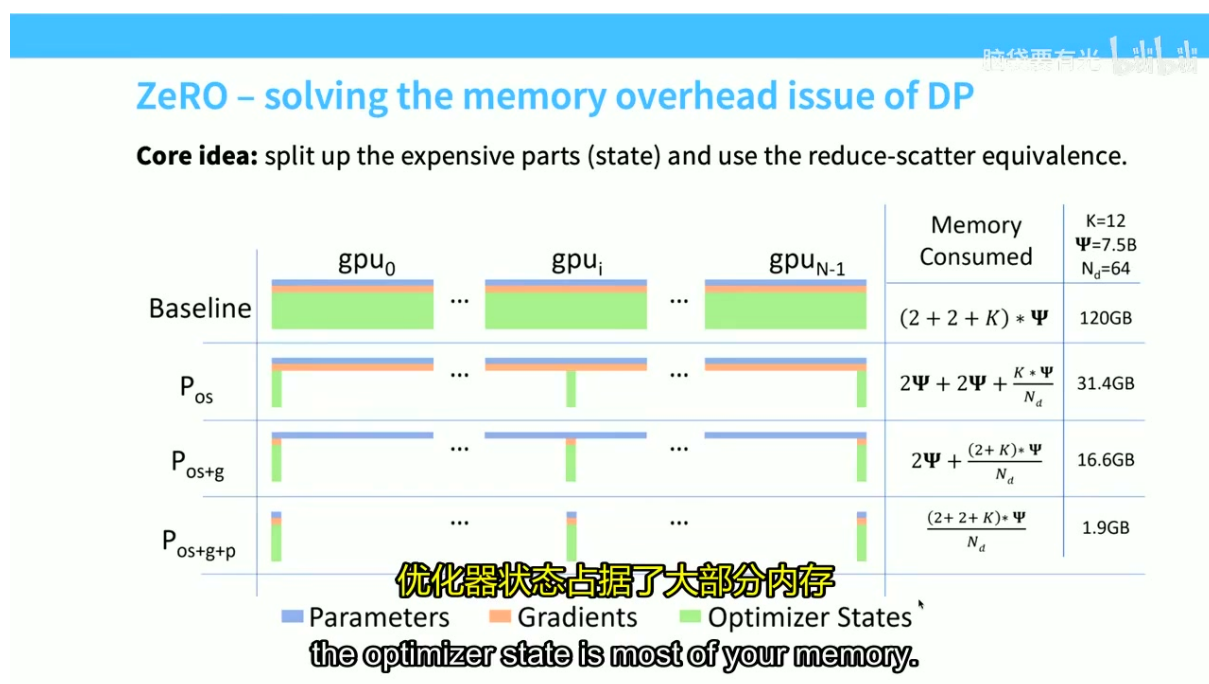


图 3: 训练过程中的内存占用分解。以 Adam 优化器为例，优化器状态（一阶矩 + 二阶矩）通常占据最大比例。³

经验法则：使用 Adam 优化器时，每个参数大约需要 **16 字节** 的存储空间（参数 2B + 梯度 2B + 一阶矩 4B + 二阶矩 4B + 其他 4B），即模型参数数量的约 8-12 倍。

3.4 ZeRO: 数据并行的内存优化

ZeRO (Zero Redundancy Optimizer) 是一系列在不增加通信开销的前提下，将优化器状态、梯度甚至参数分片到多个 GPU 上的技术。它分为三个阶段：

³ 视频画面时间区间：00:15:27-00:15:31。

3.4.1 ZeRO Stage 1: 优化器状态分片

ZeRO Stage 1 的核心思想是：每个 GPU 只保留自己负责更新的那部分参数的优化器状态。

- 所有 GPU 仍保留完整的参数和梯度；
- 优化器状态按参数切片分布到各 GPU；
- 每个 GPU 只更新自己负责的参数切片。

工作流程：

1. 每个 GPU 计算完整梯度；
2. 执行 **ReduceScatter**：将梯度切片分发到对应 GPU（每个 GPU 只获得自己需要的梯度分片）；
3. 各 GPU 用本地优化器状态更新自己负责的参数切片；
4. 执行 **AllGather**：将更新后的参数收集回所有 GPU。

ZeRO Stage 1 的通信成本

Stage 1 执行了 ReduceScatter + AllGather，其总通信量与一次 AllReduce 完全等价。因此，**Stage 1 在不增加任何通信开销的前提下，将优化器状态内存降低了 M 倍**（ M 为 GPU 数量）。这是”免费的午餐”。

3.4.2 ZeRO Stage 2: 梯度分片

在 Stage 1 的基础上，Stage 2 进一步将梯度也进行分片：

- 反向传播时不再计算完整梯度向量；
- 边计算边将梯度通过 ReduceScatter 发送给对应的工作节点；
- 一旦某层梯度不再需要，立即释放内存。

Stage 2 的实现技巧

Stage 2 的关键在于**逐层反向传播与通信重叠**：在遍历计算图时，每算完一层的梯度就立即做 ReduceScatter，而不是等所有梯度都算完再一次性通信。这利用了梯度计算的局部性——反向传播中较早层的梯度在发送后即可释放，不会阻塞后续计算。

与 Stage 1 类似，Stage 2 的通信总量仍然与基础 AllReduce 相当，因此也是”几乎零成本”的内存节省。

3.4.3 ZeRO Stage 3: 参数分片 (FSDP)

ZeRO Stage 3（在 PyTorch 中称为 FSDP，Fully Sharded Data Parallel）是最激进的内存优化：每个 GPU 在同一时间只保存参数、梯度和优化器状态的一个切片。

工作流程（逐层推进）：

1. 前向传播某层前，通过 AllGather 收集该层所需的完整权重；

2. 完成该层前向传播后，立即释放这些权重；
3. 反向传播该层前，再次 AllGather 收集权重；
4. 计算梯度后，通过 ReduceScatter 将梯度分片发送给对应节点；
5. 参数更新后，再次 AllGather 同步更新结果。

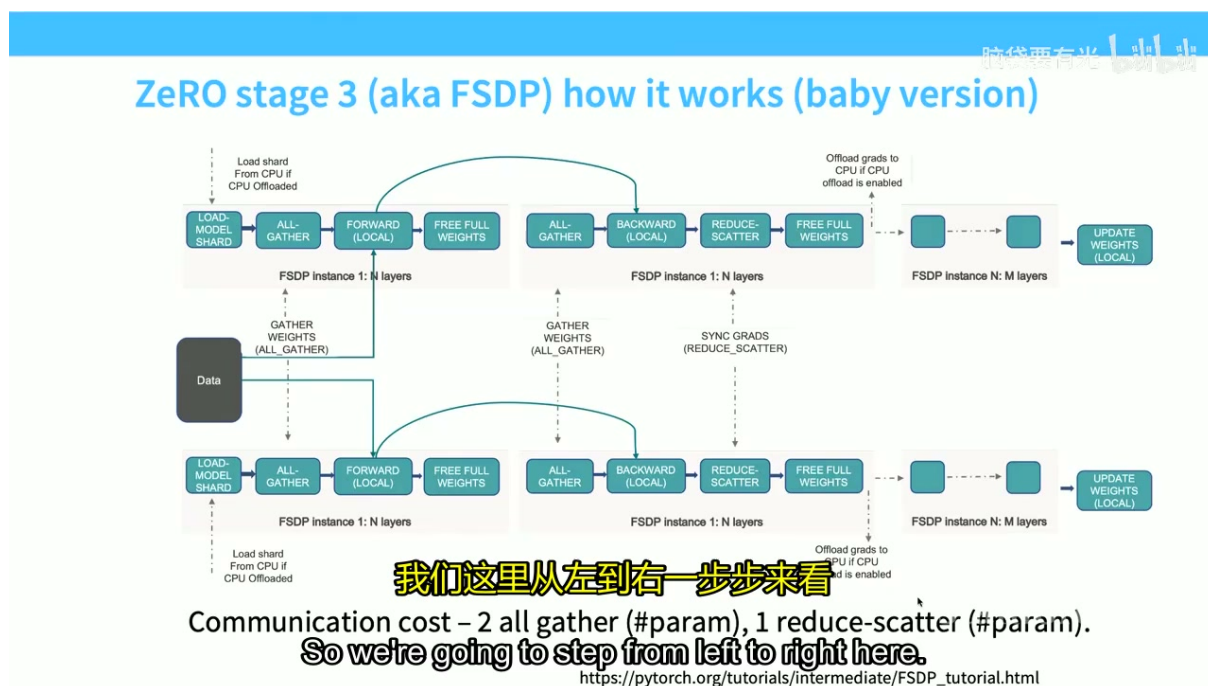


图 4: ZeRO Stage 3 (即 FSDP) 工作流程。每个 GPU 只保存参数分片，前向传播前通过 AllGather 收集完整权重，计算后立即释放。⁴

Stage 3 并非完全免费

与 Stage 1/2 不同，Stage 3 需要在前向和反向传播中多次执行 AllGather，通信量有所增加。但由于参数分片将单卡内存需求降到了最低，它使得用更多小显存 GPU 训练超大模型成为可能。这是以通信换内存的显式权衡。

3.5 批量大小与收敛性的权衡

数据并行在扩展 GPU 数量的同时，也隐式地改变了有效批量大小：

- 设单卡批量为 b ， M 个 GPU 的总批量为 $B = M \cdot b$ ；
- 更大的批量可以更好地利用硬件算力，但可能影响收敛性；
- 实践中通常采用学习率 warm-up 和线性缩放规则来适配大批量训练。

⁴ 视频画面时间区间：00:21:25–00:21:29。

大批量训练的稳定性

当全局批量达到数万甚至数十万级别时，标准 SGD 的噪声特性会发生变化。常用的应对策略包括 LARS、LAMB 等分层自适应学习率优化器，以及逐步增大批量的**渐进式批量缩放** (Progressive Batch Scaling)。

3.6 本章小结

数据并行是分布式训练最基础的并行维度：它将数据分布到多个设备，通过 AllReduce 同步梯度，实现近乎线性的计算扩展。然而标准数据并行不节省内存，每个设备仍需保存完整模型。ZeRO 优化器通过分片优化器状态 (Stage 1)、梯度 (Stage 2) 和参数 (Stage 3)，在不增加或少量增加通信开销的前提下，大幅降低了单卡内存需求。其中 Stage 1 和 Stage 2 利用了 ReduceScatter + AllGather 与 AllReduce 的通信等价性，实现了“免费”的内存优化；Stage 3 则通过逐层按需收集参数，以额外通信换取极致的内存节省，使得超大规模模型的数据并行训练成为可能。

4 张量并行 (Tensor Parallelism)

4.1 动机与核心思想

在数据并行 (Data Parallelism) 和 FSDP (Fully Sharded Data Parallel) 中，每个加速器 (GPU) 仍然需要完整地执行前向和反向传播，只是通过切分参数来降低单卡的内存占用。然而，当模型规模继续增大时，仅靠切分参数已不足以解决问题——我们还需要更细粒度地切分**计算本身**。

张量并行的本质

张量并行 (Tensor Parallelism, TP) 的核心思想是：将模型某一层内部的**矩阵乘法**沿某个维度切分成多个子矩阵，分发到不同设备上并行计算，最后通过集合通信原语合并结果。与 FSDP 切分**参数**不同，张量并行切分的是**层内张量**，因此计算过程中需要传递的是**激活值 (activations)** 而非参数。

张量并行与流水线并行 (Pipeline Parallelism) 同属**模型并行 (Model Parallelism)** 的范畴，但二者的切分维度不同：

- **流水线并行**：沿模型的**深度**方向切分（不同层放在不同设备上）。
- **张量并行**：沿模型的**宽度**方向切分（同一层内的矩阵被切分）。

4.2 列切分与行切分

在 Transformer 架构中，张量并行通常作用于 MLP 和注意力层的线性投影矩阵。具体有两种切分方式：**列切分 (Column-wise)** 和**行切分 (Row-wise)**。

4.2.1 列切分 (Column-wise Parallelism)

考虑一个线性变换 $Y = X \cdot W$ ，其中 $X \in \mathbb{R}^{B \times S \times H_{in}}$ ， $W \in \mathbb{R}^{H_{in} \times H_{out}}$ 。若将权重矩阵 W 沿**输出维度**（列方向）切分为 $W = [W_1, W_2]$ ，则：

$$Y = X \cdot W = [X \cdot W_1, X \cdot W_2]$$

每个设备只需计算 $X \cdot W_i$ ，得到部分输出。由于输出沿列方向被切分，**无需通信**即可得到各自的结果。在 Transformer 中，MLP 的上投影（up-projection）和注意力层的 Q/K/V 投影通常采用列切分。

4.2.2 行切分 (Row-wise Parallelism)

若将权重矩阵 W 沿**输入维度**（行方向）切分为 $W = \begin{bmatrix} W_1 \\ W_2 \end{bmatrix}$ ，同时将输入 X 也对应切分为 $X = [X_1, X_2]$ ，则：

$$Y = X \cdot W = X_1 \cdot W_1 + X_2 \cdot W_2$$

每个设备计算 $X_i \cdot W_i$ 后，需要通过 **AllReduce**（求和）合并最终结果。在 Transformer 中，MLP 的下投影（down-projection）和注意力层的输出投影通常采用行切分。

哪些操作不切分？

并非所有层都需要切分。以下操作通常在每台机器上**完全复制**：

- 层归一化 (Layer Normalization)
- 非线性激活函数（如 GELU、ReLU）
- Dropout
- MoE 中的路由器 (Router)

这些操作的计算量很小，切分它们带来的通信开销远大于收益。

4.3 前向与反向传播的对偶性

张量并行中一个关键的设计要点是**前向传播与反向传播的对偶性**。

设张量并行模块的前向计算可抽象为：

- F ：输入端的变换（通常为恒等映射，即复制输入）。
- 并行计算部分（矩阵乘法）。
- G ：输出端的合并操作（通常为 AllReduce 求和）。

在前向传播中：

- $F = \text{Identity}$ （将输入 X 复制到所有并行设备）。
- $G = \text{AllReduce}$ （将各设备的局部结果求和）。

在反向传播中，这一关系**恰好反转**：

- G 变成恒等映射（梯度从输出端直接传回）。
- F 变成对梯度求和（AllReduce），以得到输入 X 处的完整梯度。

实现张量并行时的常见陷阱

如果不注意前向/反向的对偶性，容易在反向传播时遗漏 AllReduce 操作，导致各设备上的输入梯度只是局部梯度而非全局梯度。正确实现时， F 和 G 的角色会在前向与反向中互换。

4.4 通信量分析

张量并行每次进行矩阵乘法时都会伴随通信操作。以典型的 Transformer MLP 为例：

- 前向传播：在列切分后、行切分前，通常需要一次 AllReduce 或 AllGather。
- 反向传播：同样需要一次 AllReduce 来聚合梯度。

通信量大致与**激活值大小**成正比，即：

$$\text{通信量} \propto B \times S \times H$$

其中：

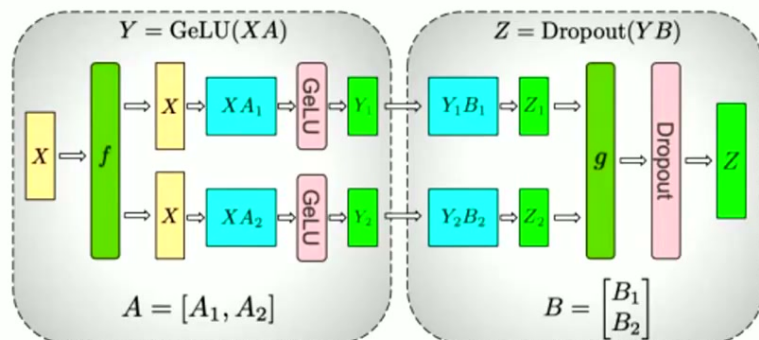
- B ：批次大小 (Batch Size)
- S ：序列长度 (Sequence Length)
- H ：隐藏层维度 (Hidden Dimension)

由于每次矩阵乘法后都可能触发通信，张量并行是**通信密集型的**。因此，它通常只在**节点内部** (intra-node) 使用——例如，一台服务器内的 8 块 GPU 通过 NVLink 高速互联。一旦跨节点 (inter-node)，通信带宽急剧下降，张量并行的效率会大幅受损。

张量并行的适用场景

- **适用**：单节点内的高速互联（如 NVLink、TPU 的 torus 网格）。
- **不适用**：跨节点的慢速网络。
- TPU 的优势：TPU 的环形网格 (torus) 拓扑使得大规模张量并行比 GPU 更容易实现高效通信。

Tensor parallel – GPUs have submatrices



Assign columns (A1, A2) and rows (B1, B2) to separate GPUs.

- In the forward pass, f is the identity, and g is an all-reduce.
- In the backward pass, f is an all-reduce, g is the identity.

实际过程就是这样

So this is a, you know, that's happening.

图 5: 张量并行中的列切分与行切分。左图为列切分 ($A = [A_1, A_2]$), 右图为行切分 ($B = [B_1; B_2]$)。

5

4.5 本章小结

- 张量并行通过切分矩阵乘法实现层内并行，是模型并行的一种。
- 列切分沿输出维度拆分，无需通信；行切分沿输入维度拆分，需要 AllReduce。
- 前向与反向传播存在对偶性，实现时需注意 F 和 G 的角色互换。
- 张量并行通信量约为 $B \cdot S \cdot H$ ，属于通信密集型，通常限制在单节点内使用。
- 与流水线并行相比，张量并行没有气泡，但通信量更大。

5 流水线并行 (Pipeline Parallelism)

5.1 基本思想

流水线并行 (Pipeline Parallelism, PP) 是最直观的模型并行方式：将模型的不同层分配到不同的 GPU 上，形成一条**流水线**。

流水线并行的定义

流水线并行将神经网络沿**深度方向**切分，每个 GPU 负责模型的一部分连续层。前向传播时，激活值从第一个 GPU 依次传递到最后一个 GPU；反向传播时，梯度从最后一个 GPU 依次传递回第一个 GPU。

例如，一个 24 层的 Transformer 被 4 个 GPU 分担：

⁵ 视频画面时间区间：00:40:14–00:40:18。

- GPU 0: 层 0–5
- GPU 1: 层 6–11
- GPU 2: 层 12–17
- GPU 3: 层 18–23

5.2 气泡问题与微批次

如果简单地让每个批次 (batch) 顺序流过流水线，会产生严重的空闲时间——即**流水线气泡 (Pipeline Bubble)**。

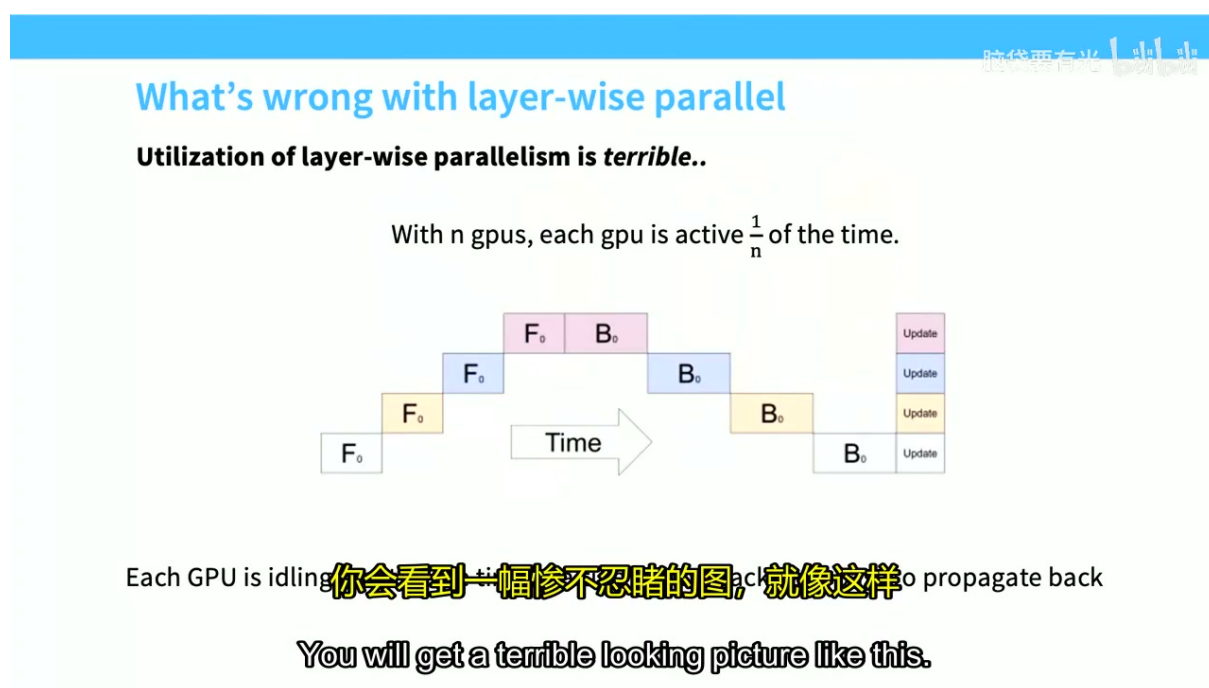


图 6: 朴素流水线并行的 GPU 利用率问题。每个 GPU 只有 $1/n$ 的时间在工作，其余时间为空闲气泡。⁶

如上图所示，在 naive 实现中：

- GPU 0 完成前向计算后，将激活值传给 GPU 1，然后进入空闲状态。
- 直到该批次的反向传播回到 GPU 0，它才重新开始工作。
- 结果：大部分时间只有一个 GPU 在工作，利用率极低。

5.2.1 微批次 (Micro-batches)

解决气泡问题的核心方法是引入**微批次 (Micro-batches)**：将一个大批次拆分为多个微批次，让它们像工厂流水线中的产品一样连续进入流水线。

假设总批次大小为 B ，拆分为 m 个微批次，每个微批次大小为 B/m 。当第一个微批次在 GPU 0 完成计算后，立即开始处理第二个微批次，同时第一个微批次的结果被传递给 GPU 1。这样，多个微批次在流水线中**重叠执行**，显著减少空闲时间。

⁶视频画面时间区间：00:32:18–00:32:22。

5.2.2 气泡占比的定量分析

气泡时间（空闲时间）与有效计算时间的比率可近似为：

$$\text{气泡占比} \approx \frac{\text{流水线阶段数}}{\text{微批次数量}} = \frac{P}{m}$$

其中：

- P ：流水线阶段数（即 GPU 数量，每个 GPU 负责一段层）。
- m ：微批次数量。

微批次数量的权衡

要彻底消除气泡，需要 $m \gg P$ ，即微批次数量远大于流水线阶段数。但这意味着**总批次大小必须非常大**，而过大的批次会导致优化效果下降（存在**临界批次大小**，超过后边际收益递减）。因此，批次大小是一种宝贵资源：它既可以用于数据并行扩展，也可以用于减少流水线气泡。

5.3 1F1B 调度策略

为了进一步优化流水线效率，研究者提出了更巧妙的调度策略。其中最具代表性的是 **1F1B (One Forward One Backward)** 调度。

5.3.1 核心思想

1F1B 的核心是**交错前向与反向计算**：在完成一定数量的前向传播后，立即开始反向传播，而不是等所有微批次的前向都完成。这样可以让后面的 GPU 尽早开始反向计算，减少空闲等待。

5.3.2 零气泡流水线 (Zero Bubble Pipeline)

更进一步的优化来自对反向传播结构的深入分析。反向传播在每个节点上需要完成两件事：

1. **反向传播梯度**：将输出端的偏导数向输入端传递（这是下游阶段依赖的，必须尽快完成）。
2. **计算权重梯度**：对当前层的参数求导（这是局部操作，不阻塞其他阶段，可以延后）。

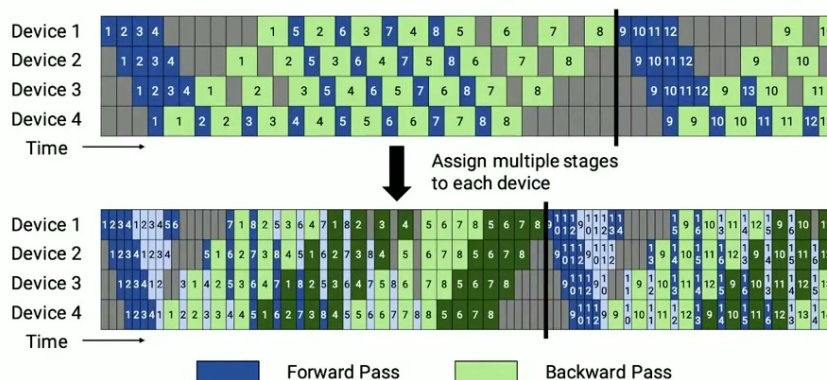
零气泡的关键洞察

将**梯度回传 (Blocking)** 与**权重梯度计算 (Non-blocking)** 分离：

- 优先完成梯度回传（标记为 B ），使下游阶段尽快开始工作。
- 将权重梯度计算（标记为 W ）推迟到后续计算间隙中执行。

通过这种调度，可以几乎完全填满流水线，实现**零气泡**。

Trading communication bandwidth for utilization



因此，人们尝试用更巧妙的方法来减少流水线的规模
Some more crazy pipeline patterns can improve utilization, but at the cost of bandwidth
ways of reducing the amount of pipeline sizes.

图 7: 流水线调度策略对比。上图展示了朴素调度的低效，下图通过增加每个设备的阶段数来提升利用率。蓝色 = 前向传播，绿色 = 反向传播。⁷

5.4 流水线并行的通信特性

与 FSDP 相比，流水线并行的通信特性非常优越：

- **通信量小**：只传递相邻层之间的激活值，数据量约为 $B \times S \times H$ 。
- **点对点通信**：只有相邻 GPU 之间需要通信，不是全对全（All-to-All）。
- **不依赖参数规模**：通信量与模型参数量无关，只与激活值大小有关。

因此，流水线并行非常适合部署在慢速网络链路上——例如，跨数据中心或跨节点的场景。

NVIDIA Megatron 的实践建议

NVIDIA 的 Megatron 并行化指南指出：如果需要在张量并行（TP）和专家并行（EP）之间做选择，**优先选择 EP**。原因是张量并行将矩阵切得太细时，会带来一系列工程复杂度（如通信频繁、负载不均等），而专家并行在 MoE 架构下通常更高效。

5.5 本章小结

- 流水线并行沿深度方向切分层，直观但容易产生**气泡**。
- 引入**微批次**可将气泡占比降至 P/m ，但需要大批次支持。
- **1F1B 调度**通过交错前向与反向计算进一步减少气泡。
- **零气泡流水线**将梯度回传与权重梯度计算分离，几乎消除空闲时间。
- 流水线并行通信量小、点对点传输，适合跨节点/慢速网络部署。

⁷ 视频画面时间区间：00:36:43–00:36:47。

6 内存优化策略

6.1 内存占用的真实构成

在大模型训练中，内存占用远不止参数本身。通过 PyTorch Profiler 的实际检测，内存使用呈现以下结构：

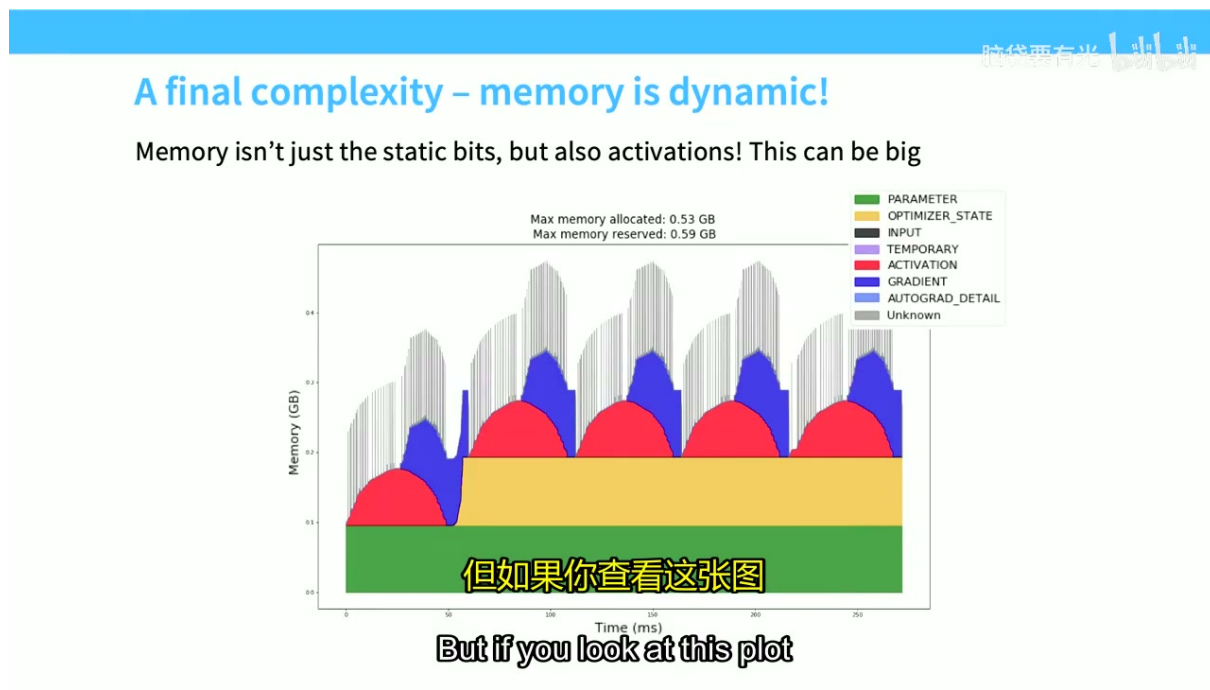


图 8: 训练过程中的内存动态占用。绿色 = 参数，黄色 = 优化器状态，红色 = 激活值。激活值往往在特定时刻形成峰值。⁸

内存占用的三大来源

1. **参数 (Parameters)**: 模型权重，通常以 FP16/BF16 或 FP32 存储。
2. **优化器状态 (Optimizer States)**: 如 Adam 的一阶矩 m 和二阶矩 v ，通常占用参数量 2-4 倍的内存。
3. **激活值 (Activations)**: 前向传播中产生的中间结果，反向传播时需要用到。这是**动态内存**，往往在训练峰值时占据最大比例。

对于现代大语言模型，当使用中等序列长度训练时，**激活值内存往往远超参数内存**。因此，任何有效的内存优化策略都必须充分考虑激活值。

6.2 激活值内存的估算

在不使用任何优化的情况下，存储全部激活值所需的内存约为：

$$M_{\text{act}} \approx 34 \cdot B \cdot S \cdot H + 5 \cdot \frac{A \cdot S^2}{H}$$

⁸ 视频画面时间区间：00:45:25–00:45:29。

其中：

- B ：批次大小 (Batch Size)
- S ：序列长度 (Sequence Length)
- H ：隐藏层维度 (Hidden Dimension)
- A ：注意力头数 (Number of Attention Heads)
- 第一项 $34 \cdot BSH$ ：主要来自 MLP 和注意力层中的线性投影。
- 第二项 $5 \cdot \frac{AS^2}{H}$ ：来自注意力机制中的二次项（如 Softmax 中间结果）。

34 · BSH 的来源

系数 34 来源于 Transformer 层中各类矩阵乘法和中间变量的累加：

- MLP 部分约占 24 个 BSH 项（上投影、下投影、激活函数输入等）。
- 注意力部分约占 10 个 BSH 项（Q/K/V 投影、输出投影等）。

具体数值会因实现细节（如是否使用 dropout、是否存储 GELU 输入等）略有差异。

6.3 张量并行对激活内存的削减

张量并行不仅可以切分参数，还能显著降低激活值内存。

- MLP 中的矩阵乘法部分（约 24 个 BSH 项）可以沿张量并行维度 T 切分，内存降至 $\frac{24 \cdot BSH}{T}$ 。
- 注意力部分的二次项（ $5 \cdot \frac{AS^2}{H}$ ）同样可以除以 T 。

然而，并非所有激活都能被切分：

- 层归一化、Dropout、残差连接的输入等操作，由于不在矩阵乘法路径上，无法通过张量并行减少内存。
- 这些未被切分的部分仍保留约 $10 \cdot BSH$ 的内存开销。

张量并行的局限

即使使用 1000 个 GPU 做张量并行，层归一化等操作的激活值仍然需要在每台设备上完整存储。因此，激活内存不会无限降低，存在一个与张量并行度无关的基础开销。

6.4 序列并行 (Sequence Parallelism)

为了进一步削减那些无法通过张量并行减少的激活内存，引入了序列并行 (Sequence Parallelism)。

序列并行的定义

序列并行不沿隐藏维度 (Hidden Dimension) 切分，而是沿序列维度 (Sequence Dimension) 切分。对于层归一化、Dropout 等轻量级操作，将输入按序列方向拆分到不同设备上，计算前通过 AllGather 收集，计算后通过 ReduceScatter 分发。

序列并行的思想与 FSDP 非常相似：

- 以**分片格式**存储激活值。
- 需要时通过 AllGather **按需重建**。
- 前向传播中 G 为 AllGather、 F 为 ReduceScatter；反向传播中二者角色互换。

结合张量并行和序列并行后，激活内存可降至：

$$M_{\text{act}}^{\text{TP+SP}} \approx \frac{34 \cdot B \cdot S \cdot H}{T}$$

如果进一步使用**激活重计算 (Activation Checkpointing)**，可以省去注意力二次项，使内存与 T 成反比。

序列并行 vs. 上下文并行

“序列并行”这个名称具有一定的误导性。实际上，**上下文并行 (Context Parallelism)** 更贴合“沿序列切分”的语义。在本讲座中，“序列并行”特指对层归一化等操作的序列维度切分策略。

6.5 激活值重计算 (Gradient Checkpointing)

激活值重计算 (Activation Checkpointing, 又称 Gradient Checkpointing) 是一种用计算换内存的经典策略。

6.5.1 基本原理

在标准反向传播中，我们需要保存每一层的输入激活值，以便计算梯度。激活值重计算的做法是：

1. **前向传播**：只保存部分**检查点 (checkpoints)**（如每层的输入），丢弃中间激活值。
2. **反向传播**：当需要某层的激活值时，**重新执行该层的前向计算**，从检查点恢复中间结果。

重计算的代价与收益

- **收益**：内存占用大幅降低（通常可减少 50%–70%）。
- **代价**：前向计算需要执行约 1.33 倍（取决于检查点策略）。
- **最佳实践**：注意力机制的重计算开销相对较小（可按块重计算，避免 $O(S^2)$ 存储）；MLP 的重计算开销较大（需要重新跑一遍完整 MLP），通常不做。

使用 Flash Attention 时，注意力部分的二次项内存可以天然避免，因为 Flash Attention 以分块方式计算，无需存储完整的注意力矩阵。

6.6 ZeRO 优化器的三阶段回顾

虽然 ZeRO (Zero Redundancy Optimizer) 在前面的数据并行部分已讨论，但从内存优化的角度，有必要系统回顾其三阶段：

表 1: ZeRO 三阶段的内存与通信对比

阶段	切分内容	单卡内存	通信开销
ZeRO-1	优化器状态	$\frac{\text{Params} + \text{Gradients}}{N} + \text{Optimizer}$	与 DDP 相同
ZeRO-2	优化器状态 + 梯度	$\frac{\text{Params}}{N} + \text{Gradients} + \text{Optimizer}$	与 DDP 相同
ZeRO-3	参数 + 梯度 + 优化器状态	$\frac{\text{Params} + \text{Gradients} + \text{Optimizer}}{N}$	额外 AllGather

FSDP 的通信隐藏技巧

PyTorch FSDP (Fully Sharded Data Parallel) 实现了 ZeRO-3 的思想, 并通过两个关键技巧使额外通信开销几乎为零:

1. **按需获取与立即释放**: 按计算图的顺序, 在需要时 AllGather 参数, 计算完成后立即释放, 最大化内存复用。
2. **通信与计算重叠**: 在对第 i 层进行前向计算的同时, 异步发起第 $i + 1$ 层的 AllGather 请求。只要计算时间大于通信时间, 通信开销就被完全隐藏。

6.7 本章小结

- 大模型训练中的内存峰值通常由**激活值**主导, 而非参数。
- 完整激活内存约为 $34 \cdot BSH + 5 \cdot \frac{AS^2}{H}$, 其中 BSH 项是优化重点。
- **张量并行**可将矩阵乘法相关的激活内存按并行度 T 线性缩减。
- **序列并行**进一步将层归一化等操作的激活内存也按 T 缩减。
- **激活值重计算**用额外前向计算换取内存, 对注意力层尤为有效。
- **ZeRO-3 / FSDP** 通过切分参数、梯度、优化器状态, 配合通信-计算重叠, 实现近零开销的内存扩展。

7 专家并行与 MoE

7.1 MoE 架构回顾与专家并行的动机

混合专家模型 (Mixture of Experts, MoE) 已成为当前大规模语言模型的标准架构之一。与传统的稠密 Transformer 不同, MoE 在每一层中将前馈网络 (MLP) 替换为多个并行的“专家”网络, 并通过一个门控机制 (Gating/Router) 为每个输入 token 选择最相关的 Top-K 个专家进行计算。这种稀疏激活的设计使得模型可以在大幅增加参数数量的同时, 保持相对稳定的计算开销。

然而, MoE 架构的引入也给分布式训练带来了新的挑战: 每个 token 需要被路由到不同的专家, 而这些专家可能分布在不同的设备上。**专家并行** (Expert Parallelism, EP) 正是为了解决这一问题而提出的并行策略——它将不同的专家分配到不同的 GPU 上, 让每个设备只负责维护一部分专家的参数。

专家并行的核心思想

专家并行将 MoE 层中的各个专家网络分散到多个设备上。对于每个输入 token，门控网络决定它应该被发送到哪些专家；系统通过 AllToAll 通信将 token 路由到目标设备，计算完成后再将结果收集回来。这与张量并行类似，都属于高带宽、低延迟的并行原语。

7.2 专家并行 vs. 张量并行：为何优先选择 EP？

在 MoE 模型中，当需要在 MLP 层进行某种并行切分时，NVIDIA 的 Megatron 并行化指南明确建议：**优先选择专家并行（EP），而非张量并行（TP）**。这一建议背后有多重系统层面的考量。

7.2.1 矩阵粒度与 GPU 利用率

张量并行会将 MLP 中的大矩阵沿隐藏维度切分为更小的子矩阵。当切分度过高时，每个 GPU 上处理的矩阵块变得非常细小，导致 GPU 的 Tensor Core 无法充分发挥计算效率——现代 GPU 在处理大块矩阵乘法时才能达到峰值算力。

相比之下，专家并行并不切分单个矩阵，而是将**完整的专家网络**分配到不同设备上。每个专家仍然处理完整维度的矩阵运算，只是处理的 token 数量变少了（因为每个 token 只路由到少数几个专家）。这保持了矩阵运算的大粒度，有利于维持高 GPU 利用率。

7.2.2 稀疏路由 vs. 密集切分

MoE 的核心优势在于**稀疏激活**：每个 token 只激活少数几个专家（如 Top-1 或 Top-2），而非全部专家。专家并行天然契合这种稀疏性——只需将 token 精确路由到它需要去的专家即可，避免了不必要的计算和通信。

而张量并行则是“密集”的：无论 token 是否需要某个专家，矩阵都被强制切分到所有设备上，所有设备都必须参与每个 token 的计算。这种密集切分与 MoE 的稀疏设计理念相悖，引入了额外的开销。

专家并行并非万能

尽管专家并行在 MoE 场景下优于张量并行，但专家并行本身仍然非常复杂且难以实现。从系统层面看，FSDP、张量并行、流水线并行已经很难高效运行，而专家并行引入了额外的路由调度、负载均衡和 AllToAll 通信挑战。不要因为它“理论上更好”就低估工程实现的难度。

7.3 专家并行的通信模式：AllToAll 与低延迟调度

专家并行的通信模式与张量并行类似，都属于需要**高带宽、低延迟**的节点内通信。每当进行 MoE 层的 MLP 计算时，系统需要执行以下步骤：

1. **路由决策**：门控网络为每个 token 决定目标专家（Top-K）。
2. **Token 分发**：通过 AllToAll 通信，将 token 的表示发送到持有目标专家的设备上。
3. **本地计算**：每个设备上的专家只处理被路由到它的 token 子集。
4. **结果收集**：再次通过 AllToAll 将计算结果传回原设备。

降低调度延迟至关重要

MoE 的通信模式与张量并行类似，但有一个关键区别：计算正在等待 token 到达。如果 token 路由延迟过高，GPU 将处于空闲等待状态，导致利用率下降。因此，降低 AllToAll 的调度延迟是专家并行系统的核心优化目标之一。

7.4 专家并行的工程挑战与前沿库

为了榨干专家并行的最后一点性能，工业界已经开发了多个高度优化的专用库。

7.4.1 DeepSeek 的 DeepSeek EP

DeepSeek 团队为 DeepSeek-V3 开发了自己的专家并行路由与分发库（内部称为类似 `deepseek_ep` 的系统）。该库深入研究了底层 GPU 网络原语，通过组合不同操作或让硬件自行处理调度，将 MoE 路由效率最大化。一个有趣的冷知识是：为了进一步加速网络通信，DeepSeek 团队甚至找到了未公开的 PTX 指令（类似于 GPU 机器码的底层指令），利用这些指令来绕过常规 CUDA API 的开销。

7.4.2 NVIDIA 的 Hybrid EP

NVIDIA 也推出了名为 `hybrid_ep` 的库，旨在通过底层硬件实现高效的专家并行调度。其基本思路与 DeepSeek 类似：在硬件层面尽可能减少调度延迟，让 AllToAll 通信与计算更好地重叠。

PTX 指令与极致优化

PTX（Parallel Thread Execution）是 NVIDIA GPU 的中间指令集架构，介于高级 CUDA 代码和最终的 SASS 机器码之间。PTX 指令通常由编译器自动生成，但高级开发者可以手写 PTX 来精确控制寄存器分配、内存访问模式和指令调度。DeepSeek 团队利用未公开的 PTX 指令来优化网络通信，体现了大规模训练系统优化已经深入到了硬件微架构层面。

7.5 专家并行与其他并行策略的组约束

到目前为止，我们一直将各种并行策略视为可以像乐高积木一样自由组合的模块。对于数据并行和张量并行，这种组合确实相对直接。但专家并行引入了一些额外的组约束，需要特别注意。

7.5.1 朴素组合：EP 与 DP 共享复制组

最直观的做法是将数据并行与专家并行结合起来，让数据并行和专家并行的复制组使用同一组设备。例如，如果有 8 路数据并行，就将 8 个专家分片到这 8 个复制组上，数据也被拆分到这 8 个组中。

这种方案的问题是：专家并行的扩展能力会有一个上限，因为它被数据并行的复制组数量所限制。同时，这也会限制数据并行与张量并行的交互方式——当专家并行的扩展受限时，整个系统的并行策略空间就被压缩了。

7.5.2 注意力层与 MLP 层的并行需求矛盾

MoE 架构有一个重要的系统现象：MoE 只影响 MLP 部分，完全不影响注意力层。因此，当我们用专家并行拆分 MLP 时，注意力层并没有被并行化。

为了并行化注意力层，我们通常需要引入张量并行。但这产生了一个奇怪的权衡：

- 注意力层需要**高张量并行度**来拆分巨大的注意力矩阵；
- MLP 层（MoE）却需要**低张量并行度**来保持矩阵粒度。

如果同时对注意力层和 MoE 层使用高张量并行，矩阵会被拆成非常细小的碎片，导致计算利用率极低。

7.5.3 解决方案：解耦的张量并行

近年来，业界提出了**解耦的张量并行**方案：注意力层使用一种张量并行配置，而 MoE 层则采用另一种（或完全不使用张量并行）。这样可以将 MLP 和注意力机制中的张量并行完全解耦，形成更复杂却更有效的并行组合。

7.6 上下文并行（Context Parallelism）

在结束专家并行话题之前，我们简要介绍**上下文并行**（也叫**环形注意力**，Ring Attention）。其基本思想是：将长序列中的激活值分散到不同加速器上，按照 TPU 的网格拓扑以环形方式将激活值传递给所需的设备。

上下文并行最早在 TPU 上被证明效果良好，现在已成为处理**长上下文扩展**和**模型推理**阶段的标准并行化策略之一。其概念与前面讲的内容有很多重叠，核心仍然是在序列维度上进行切分和通信。

7.7 本章小结

- 专家并行（EP）将 MoE 的不同专家分配到不同设备，天然契合 MoE 的稀疏激活特性。
- 在 MoE 模型中，EP 优于 TP，因为 EP 保持了矩阵运算的大粒度，避免了密集切分带来的开销。
- EP 的核心通信原语是 AllToAll，降低调度延迟是系统优化的关键。
- 前沿系统（如 DeepSeek EP、NVIDIA Hybrid EP）已深入到底层 PTX 指令级别来优化 EP 性能。
- EP 与 DP 的朴素组合存在扩展上限；注意力层和 MLP 层对 TP 的需求矛盾需要通过**解耦 TP**来解决。
- 上下文并行（环形注意力）是处理超长序列的重要补充策略。

Recap: LLM parallelism table

Method	Comm/Sync	Param memory per rank	Activation / KV memory per rank	Main bandwidth cost	Scales global batch?	Easy to use?
DDP / ZeRO-1	gradient all-reduce per step	No param scaling (ZeRO-1 only opt state)	None	Gradient traffic ~ $O(\text{params})$	Yes, linear in DP	Very
FSDP / ZeRO-3	Gradient all reduce - can be overlapped	~1/DP for params/grads/optimizer state	None	Param traffic ~ $O(\text{params})$, higher than DDP, overlap	Yes, linear in DP	Moderate
Pipeline parallel	Activation between layers, pipeline bubbles	~1/PP	Depends on pipeline buffers	Activation traffic between stages	No, but needs microbatches	Hard
Tensor parallel	Blocking activation communication	~1/TP for TP-sharded weights	~1/TP for relevant matmul activations w/ SP	Activation-sized collectives every block	No	Hard
Sequence / Context parallel	per-layer sequence-shard exchange	None	~1/SP or 1/CP on sequence-side activations / KV	Activation / KV communication	No	Hard
Expert parallel (MoE)	token dispatch - all-to-all per MoE	~1/EP for expert weights only	None	Token-routing all-to-all	No, but needs enough tokens per	Hard

我整理了一张并行化策略的大表格，包括数据并行
I've made a big table of parallelization strategies, including data parallel

图 9: 并行化策略对比表格，总结各策略的优缺点与适用场景。⁹

8 注意力层并行化与综合策略

8.1 从单一策略到多维组合

前面各节分别介绍了数据并行（DP/FSDP）、张量并行（TP）、流水线并行（PP）、专家并行（EP）和序列/上下文并行。在真实的大规模训练中，这些策略很少单独使用——它们被组合成所谓的 3D 并行或 4D 并行系统。

多维并行的核心目标

将所有并行策略整合起来，在不同通信拓扑下让计算单元保持充分利用。关键在于：**只要计算时间比通信时间长，原则上就可以将通信开销隐藏在计算过程中。**

8.2 计算-通信权衡的定量分析

并行化有一个非常实用的定量分析框架。对于给定的批次大小和并行化策略，我们可以计算：

1. 每层产生多少**计算量**；
2. 每层产生多少**通信量**；
3. 计算时间与通信时间的**比值**。

如果计算时间大于通信时间，系统处于**计算受限**的高效区域；反之则进入**通信受限**的性能瓶颈区。

⁹视频画面时间区间：01:02:00–01:02:04。

Model vs Tensor parallel (TPU book)

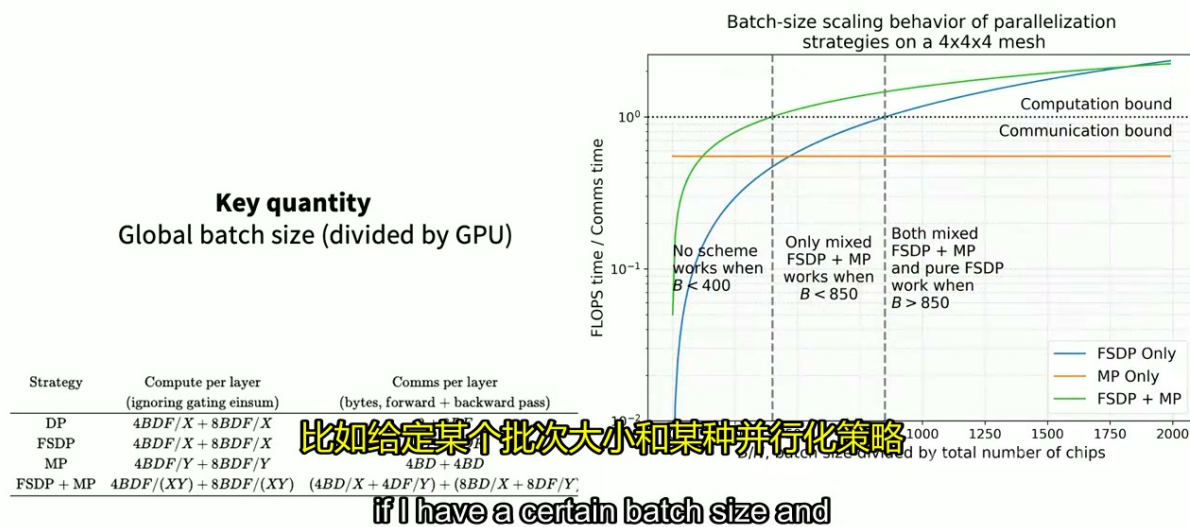


图 10: GPU 利用率 vs 批次大小曲线。虚线以上为计算受限的高效区域，虚线以下为通信受限的瓶颈区。¹⁰

8.2.1 FSDP 的扩展极限

当批次大小足够大时（例如每个芯片的批次大小达到 2000），仅使用 FSDP 就可以获得出色的扩展性——系统完全受限于计算，通信开销被完美隐藏。但随着批次大小的减小，仅使用 FSDP 就会达到通信受限的临界点。

8.2.2 叠加策略让曲线右移

为了在小批次大小下继续保持高效，需要引入模型并行（张量并行）。每叠加一种策略，高效区域的边界就会向右移动——即在更小的批次大小下也能保持计算受限。这就是 3D/4D 并行的本质：通过策略叠加来扩展高效区域。

8.3 实操指南：如何选择并行策略

尽管并行化看起来复杂，但讲师总结了一套非常简单的经验法则：

¹⁰ 视频画面时间区间：01:04:35–01:04:39。

并行策略选择的实操指南

1. **尽量减少模型并行，尽量增加数据并行**——尽可能用大部分 GPU 来切分数据。
2. **高速互联限制在节点内**（NVLink 范围，通常一台机器 8 张 GPU）：
 - 使用张量并行（TP）或专家并行（EP），规模不超过 8。
3. **解决模型剩余部分的适配**：
 - 用流水线并行（PP）或 ZeRO-3 来跨节点拆分模型。
4. **一旦模型能放得下，剩下的全部用数据并行（FSDP）。**
5. 如果最终批量大小太小，用**梯度累积**来提高 GPU 利用率。
6. 如果是 MoE 模型，优先用**专家并行**代替张量并行。
7. 如果处理长序列，用**上下文并行**（环形注意力）。

8.3.1 序列并行的定位

有同学提问：为什么推荐中没有单独列出**序列并行**？讲师的回答是：序列并行通常与张量并行一起使用，用来减少激活值内存。它更像是一个**降低激活内存的附加模块**，大多数情况下并不是一个独立的并行维度。

8.4 Megatron 论文的量化证据

NVIDIA 和斯坦福大学合作的 Megatron 论文提供了很好的量化证据，展示了并行策略应如何随规模变化：

- **数据并行**先用到极致；
- **张量并行**慢慢增加到 8，然后就不再加了——因为超过 8 就没有意义；
- 从那时起，**流水线并行**持续增加；
- 当规模足够大时，数据并行反而会减少（因为只需足够的 TP 和 PP 就能装下模型），**剩下的资源全部用于数据并行**。

Scaling strategies from Narayanan 2021

Efficient Large-Scale Language Model Training on GPU Clusters
Using Megatron-LMDeepak Narayanan^{1*}, Mohammad Shoeybi¹, Jared Casper¹, Patrick LeGresley²,
Mostafa Patwary¹, Vijay Korthukanti¹, Dmitri Vainbrand¹, Prithvi Kashinkunti¹,
Julie Bernauer³, Bryan Catanzano³, Amir Phanishayee¹, Mateo Zaharia¹
¹NVIDIA ²Stanford University ³Microsoft Research

Number of parameters (billion)	Attention heads	Hidden size	Number of layers	Tensor model-parallel size	Pipeline model-parallel size	Number of GPUs	Batch size	Achieved teraFLOP/s per GPU	Percentage of theoretical peak FLOP/s	Achieved aggregate petaFLOP/s	DP size
1.7	24	2304	24	1	1	32	512	137	44%	4.4	32
3.6	32	3072	30	2	1	64	512	138	44%	8.8	32
7.5	32	4096	36	4	1	128	512	142	46%	18.2	32
18.4	48	6144	40	8	1	256	1024	135	43%	34.6	32
39.1	64	8192	48	8	2	512	1536	138	44%	70.8	32
76.1	80	10240	60	8	4	1024	1792	140	45%	143.8	32
145.6	96	12288	80	8	8	1536	2304	148	47%	227.1	24
310.1	128	16384	96	8	16	1920	2160	155	50%	297.4	15
529.6	128	20480	105	8	35	2520	2520	163	52%	410.2	9
1008.0	160	25600	128	8	64	3072	3072	163	52%	502.0	6

Notes

- Tensor parallel **你的并行策略应该怎么跟着规模变化**
- Pipeline parallel **how should your parallelization strategies**
- Data parallel gradually **change across different scales** model having DP=6

图 11: Megatron 论文中的并行策略随 GPU 数量变化的图表。随着 GPU 规模扩大, 最优策略从 DP 主导过渡到 TP+PP 主导。¹¹

8.4.1 张量并行的最优规模

论文明确显示: 张量并行度设为 8 是最优的, 一旦超过这个值就会出问题。这是因为:

- 超过 8 后, 节点间通信带宽急剧下降;
- 矩阵被切得太细, GPU 利用率降低。

8.4.2 流水线并行对批次大小的要求

流水线并行需要较大的批次大小才能有效利用机器。流水线并行规模越大, 不同批次大小下的性能差距 (气泡效应) 就越明显。

8.4.3 激活重计算的反直觉价值

另一个重要结论是: 你应该进行大量的激活值重计算。这起初是一个非常反直觉的结论——通过重计算来增加计算量, 反而能获得更好的利用率。原因是:

1. 激活重计算节省内存;
2. 节省的内存可以转化为更大的批次大小;
3. 更大的批次大小带来更好的利用率。

¹¹ 视频画面时间区间: 01:10:13–01:10:17。

不要低估激活重计算的价值

很多人误以为“激活重计算”浪费了计算”。实际上，在内存受限的场景下，重计算是一种用计算换内存、再用内存换批次大小的优雅策略，最终往往能够提升整体吞吐量。

8.5 本章小结

- 多维并行（3D/4D）的核心目标是将通信开销隐藏在计算过程中。
- 可以通过定量分析（计算量/通信量比值）来判断系统是否处于高效区域。
- 实操指南：最小化模型并行、最大化数据并行；TP/EP 限制在节点内（ ≤ 8 ）；跨节点用 PP；剩余用 DP。
- Megatron 论文提供了量化证据：TP 最优为 8，PP 需要大批次，DP 应尽可能大。
- 激活重计算是反直觉但高效的策略：用计算换内存，再用内存换批次大小。

9 实际大规模训练案例与总结

9.1 案例研究：不同模型的并行策略

通过观察真实模型的训练配置，我们可以了解现代并行策略在实际中是如何应用的。

9.1.1 OLMo：纯 FSDP 的小模型训练

OLMo 是一个 70 亿参数的开源模型，完全使用 **FSDP** 在大量加速器上训练。这表明：**对于小模型（7B 参数），FSDP 的扩展性惊人**，即便是在大量 GPU 上也能高效运行。FSDP 是一个非常不错的中小模型并行训练策略。

9.1.2 DeepSeek V1 与 V3：从稠密到 MoE 的演进

- **DeepSeek V1**：仅使用数据并行（ZeRO）、张量并行、序列并行和流水线并行——经典的稠密模型并行组合。
- **DeepSeek V3**：MoE 模型，不仅使用了流水线并行，还采用了**专家并行**（而非张量并行）来拆分 MLP 层。

DeepSeek V3 的专家并行非常特别：**64 路专家并行**，将 8 台机器组成一组作为专家并行域。为了实现这种大规模的专家并行，他们沿用了流水线并行中的微批次调度技巧，确保专家并行不会出现低利用率的时段。

9.1.3 Qwen：经典的三维并行

Qwen（通义千问）采用了经典的 **ZeRO + 张量并行 + 流水线并行** 组合。这是非常合理的标准配置：数据并行处理不同数据，张量并行拆分注意力层，流水线并行跨节点拆分层。

9.1.4 LLaMA 3：超大规模稠密模型的标杆

LLaMA 3 405B 是一个巨大的**稠密模型**（非 MoE），其训练报告详细拆解了各阶段的并行策略：

表 2: LLaMA 3 405B 预训练阶段的并行配置

并行维度 & 配置
张量并行度 & 8
上下文并行度 & 1（预训练阶段）→ 提高（长上下文扩展）
流水线并行度 & 16
数据并行度 & 128

在长上下文扩展阶段，他们**提高了上下文并行度**、**减少了数据并行度**，以并行处理这个非常吃内存的阶段。注意这里采用的是**张量并行**而非专家并行，因为 LLaMA 3 是稠密模型。

基础设施的现实挑战

假如你成为这类公司的基础设施工程师，会发现 GPU 经常出故障。据说在 LLaMA 3 405B 模型训练期间，GPU 故障了 **148 次**。因此，大规模训练不仅需要高效的并行策略，还需要冗余机制来应对训练中的各种故障——这也是分布式系统的重要挑战。

9.1.5 Gemma 2：TPU 上的简洁方案

Gemma 2 是谷歌的开源模型，仅使用了 **FSDP + 张量并行 + 序列并行**。这验证了谷歌的说法：**对于 TPU，你真的不需要做流水线并行**——只需一个足够大的环形网格，在这个大网格网络上进行张量并行就足够了。这种方案能否无限扩展尚不清楚，但至少在 Gemma 2 的规模上是有效的。

9.1.6 Mixtral 8x22B：MoE 的经典配置

Mixtral 8x22B 采用了：

- 8 路专家并行
- 4 路流水线并行
- 4 路张量并行（专门用于注意力层）

这符合”专家并行数量大致保持在 8 左右”的经验法则。张量并行专门用于注意力层，而 MLP 层则通过专家并行处理——这正是前面提到的**解耦张量并行**的实践。

9.1.7 其他前沿模型

- **Nemotron-3 Super**：效仿 DeepSeek V3，采用大量专家并行 + 上下文并行。
- **Chameleon**：32 路专家并行 + 8 路流水线并行 + 2 路张量并行（用于注意力矩阵）。

9.2 NVIDIA Megatron-LM 的推荐配置

NVIDIA 在 `megatron-lm` 代码库中发布了大量针对不同模型规模和设置的推荐训练配置。感兴趣的同学可以查看他们为 Mixtral 8x22B、LLaMA 或 DeepSeek V3 等模型建议的配置。即使在张量并行内部，不同的子配置（如行切分 vs 列切分的组合方式）也会显著影响整体性能，因此**系统集成至关重要**。

9.3 各模型配置的共同规律

观察所有这些实际案例，可以总结出以下共同规律：

实际训练配置的共性

1. **尽可能使用数据并行**——所有模型都将数据并行域扩展到极致。
2. **张量并行几乎总是限制在 8 以内**——超过 8 后收益递减、问题增加。
3. **专家并行现在有时会规模很大**（如 32 路、64 路），这主要归功于 DeepSeek V3 等团队积累的大规模 EP 训练经验。
4. **稠密模型用 TP 拆分注意力层，MoE 模型用 EP 拆分 MLP 层**——两者通过**解耦 TP** 协调。

10 总结与延伸

10.1 讲座核心要点回顾

本次讲座的核心要点是：我们需要考虑**多 GPU、多节点乃至多数据中心**的并行化。为了在这种体系下提高效率，必须动用所有手段——因为你拥有高速链路（NVLink），也有慢速的网络连接（以太网）；有些技术会利用批次大小，有些会利用模型结构。你总想尽可能把它们都用满。

10.2 核心机制的结构化提炼

表 3: 并行策略的核心机制对比

策略 & 切分对象 & 通信原语 & 适用场景
数据并行 (FSDP) & 数据批次 & AllReduce / AllGather & 模型能装下时优先使用
张量并行 (TP) & 单层内的参数 & AllGather / ReduceScatter & 节点内高速互联, ≤ 8
流水线并行 (PP) & 模型的不同层 & 点对点通信 & 跨节点扩展, 需要大批次
专家并行 (EP) & MoE 的不同专家 & AllToAll & MoE 模型, 替代 TP 拆分 MLP
上下文并行 (CP) & 序列维度的激活 & 环形通信 & 长上下文扩展
序列并行 (SP) & 序列维度的残差 & AllGather / ReduceScatter & 配合 ³ 4TP 降低激活内存

10.3 各节之间的交叉联系

- **第 1–2 节**的硬件拓扑知识（TPU 环形网络 vs GPU 交换机拓扑）决定了**第 3–5 节**中各并行策略的通信效率上限。
- **第 6 节**的内存优化（激活重计算、ZeRO、序列并行）直接影响了**第 8–9 节**中可使用的批次大小，进而影响并行策略的选择。
- **第 7 节**的专家并行与**第 4 节**的张量并行形成竞争关系——在 MoE 模型中 EP 优于 TP，但两者需要通过**解耦 TP**来协同工作。
- **第 8 节**的计算-通信定量分析框架，是理解**第 9 节**中各模型为何选择特定配置的关键工具。

10.4 实践启示与下一步

1. **从简单开始**：小模型（ $\leq 7B$ ）优先尝试纯 FSDP，很多情况下已经足够。
2. **节点内优先 TP/EP**：如果单节点装不下，先在节点内（8 卡）引入张量并行或专家并行。
3. **跨节点用 PP**：当需要扩展到多节点时，引入流水线并行。
4. **最大化 DP**：模型能装下后，将所有剩余 GPU 用于数据并行。
5. **善用重计算**：不要忌讳激活重计算——它往往能通过释放内存来提升批次大小，最终提高吞吐量。
6. **参考 Megatron 配置**：NVIDIA 的 megatron-lm 仓库提供了大量经过验证的推荐配置，是实际调参的重要参考。
7. **关注故障恢复**：大规模训练不仅是性能问题，也是可靠性问题——GPU 故障是常态，需要冗余和检查点机制。

10.5 展望

讲座最后提到，下节课将讲解**缩放法则**（Scaling Laws）——这是理解模型规模、数据量和计算量如何共同决定模型性能的关键理论框架。缩放法则与并行化策略紧密相关：只有高效的并行系统，才能支撑起缩放法则所要求的大规模训练。

从并行化到缩放法则

并行化解决的是“如何在给定硬件上高效训练大模型”的问题；缩放法则回答的是“应该训练多大的模型、用多少数据”的问题。两者结合，才能指导实际的训练决策：先通过缩放法则确定目标规模，再通过并行化策略设计来实现该规模的训练。